

Welcome to Half Way through Python Bytes

This little booklet will give you some Python to think about and code to write in between club meetings. If you would like more challenges: Talk to the coaches!

On this start page we have listed five programs for you. Writing them will check your Python skills so far. If you do not know how to do them: Don't worry! That's what the Python Bytes club and the coaches are for.

Coding Exercise 1: A simple mad-lib. Use the Python built-in function `input()` to ask for five pieces of information: A *name*, a *food item*, a *number*, a *toy*, and an *occupation*. Then print the mad-lib, filling in the blanks from the inputs: `One day _____ was finishing up a _____ when they had a great idea! They entered a nearby skyscraper, ran up the _____ stairs to the top and threw a _____ off the roof. It floated back down on a gust of wind and lightly bumped into a _____. No injuries were reported.`

Coding Exercise 2: How many characters? Define a string variable called `s` to have the value `"On day one I saw a bee."` Use the built-in function for length, called `len()`, to print the number of characters in this string. Your program should print out `"The string s contains _____ characters."`

Coding Exercise 3: Divisible by 17. Write a program that counts from 1 to 300. For each number: Check to see if each number is divisible by 17. If it is divisible by 17: Print the message `"_____ is divisible by 17."` You may find it helpful to use the remainder operator. In python this is the percent sign `%`. You can see how this remainder operator works by running the Python code `print(9 % 3, 9 % 2, 9 % 5)`.

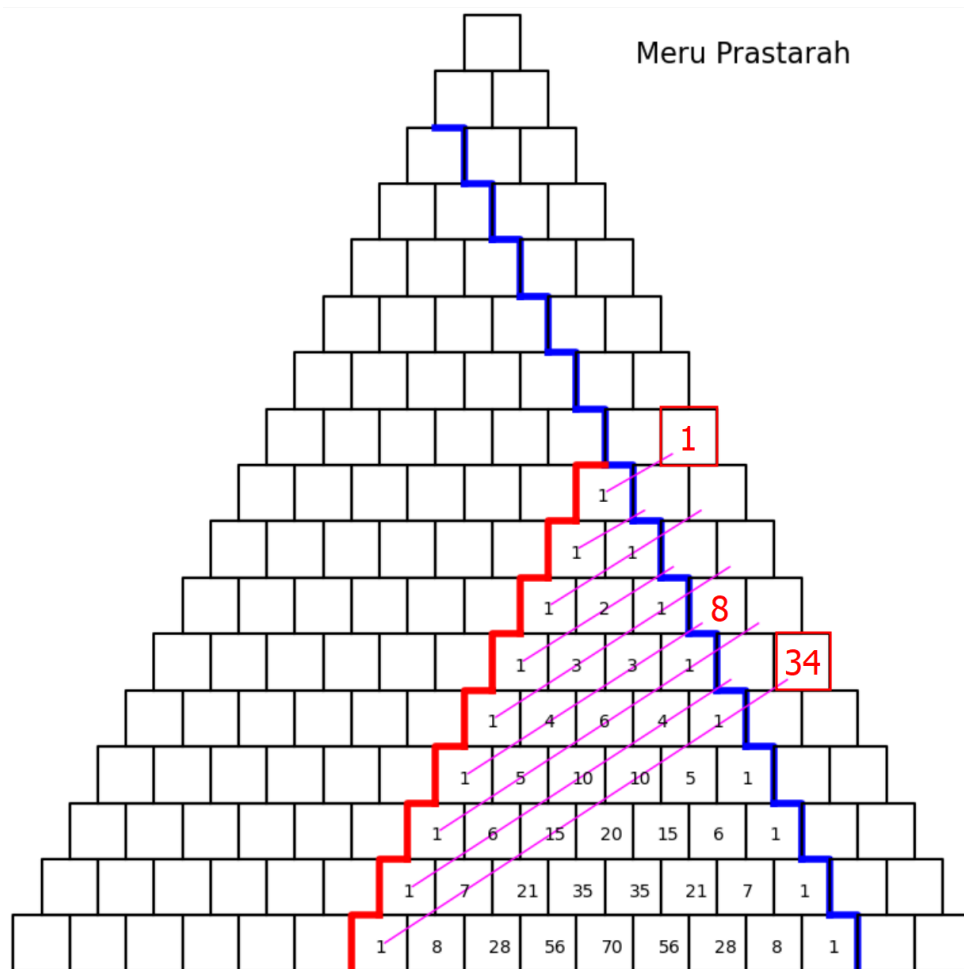
Coding Exercise 4: Fibonacci. Write a program that calculates and prints the first 50 numbers of the Fibonacci sequence. The first seven of them are: 0, 1, 1, 2, 3, 5, 8.

Coding Exercise 5: Random falling. Use the import line `from random import randint` to make the 'random integer' function available to your program. Create a string which is a random number of spaces followed by the letter X. You can do this by simply multiplying the space character `" "` by an integer like `7` and then adding `+ "X"`.

Remember: These exercises are a good check of your Python skills. If you are not sure how to do something: Ask your coaches for a little help.

Below is a drawing of Mount Meru, the mystic mathematical mountain at the center of the universe (so I am told). This mountain has other names as well (including “the Pascal triangle”) and fortunately for us the mountain has stairs built in so we can go up and down as we like.

Extra challenge: Three numbers are filled in on the right slopes of Mount Meru in red ink. They are 1, 8 and 34. Figure out how these numbers are connected to the numbers in black; and then fill in the missing numbers. Hint: The diagonal lines (magenta) are a *guide*.



Extra Extra Challenge: Can you figure out a way of continuing the red numbers all the way up to the top of the diagram?

Fibonacci ratios

Imagine a Fibonacci number, like say 5. This is followed by another number, namely 8. Let's make a fraction from them, called a Fibonacci *ratio*: It will be the next number (8) divided by the first number (5). The start number for Fibonacci is 0. The next number is 1. Add them to get the third number: 1 again. Now we have 0, 1, 1. Add 1 + 1 to get 2. Now we have 0, 1, 1, 2. Repeat! Now it's 0, 1, 1, 2, 3. Again! Now it's 0, 1, 1, 2, 3, 5... Let's calculate some Fibonacci ratios:

$$1/1 = 1$$

$$2/1 = 2$$

$$3/2 = 1.5$$

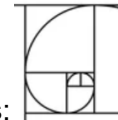
$$5/3 = 1.666666666666666666666666666666$$

$$8/5 = 1.6$$

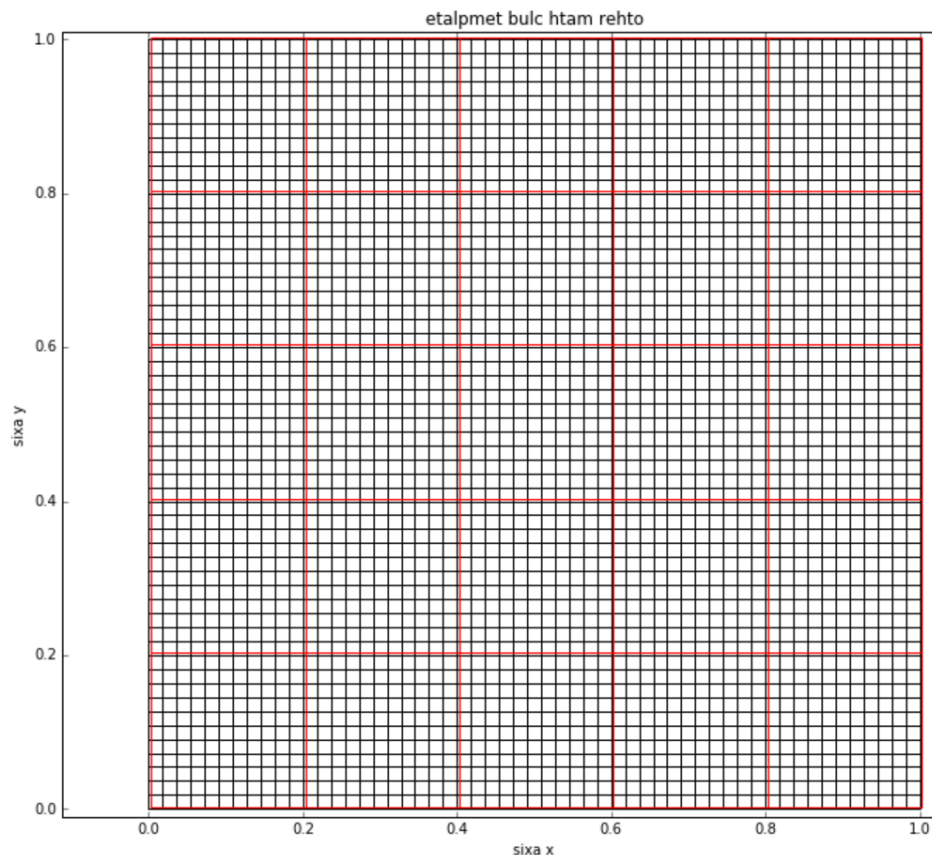
What happens to the ratios as you keep going on to more Fibonacci numbers?

Coding challenge: Write a program to print out the first 50 Fibonacci ratios.

Advanced coding challenge: Do the same starting with numbers *other than* 0 and 1. Compare the result with the Fibonacci ratio result.



Reward yourself by drawing a Fibonacci spiral. This starts out like this:



Graphing Grammar

This is a coding project. It will be challenging!

This Python code `s = "toothpaste"` means that the variable `s` contains a string of letters. This project begins with a very simple string. We will modify it to make it longer. Finally we read the result as *drawing instructions* for a turtle. For this we will need two rules. But first here is a warm-up exercise: Write a Python program that inputs a string and then prints every *other* letter of that string. Begin with this code:

```
s = input('Enter some text: ')
for c in s:
    print(c)
```

Onward: Let us begin with a simple string: `s = "x"`. `"x"` is both a word and a (short) sentence. To build longer sentences use **Rule S**: Go through the string one character at a time and build a new sentence. If the character in the starting string is `"+"` or `"-"` simply copy it into the new sentence. If the character is `"x"` or `"y"`: Replace it in the new sentence like so:

- `"x"` -> `"y+x+y"`
- `"y"` -> `"x-y-x"`

That is, every place you find `x` replace it with `y+x+y`. Doing this to `"x"` will produce a new sentence that is just `"y+x+y"`. Now apply Rule S again, to `"y+x+y"`. Now we end up with `"x-y-x+y+x+y-x-y-x"`. Our sentences are growing longer. Repeat Rule S three more times, each time making our result sentence longer. The end result will be quite long, almost 500 characters. Now to draw the end result using a turtle! How to do this? Answer: **Rule D**: Step through the string one character at a time. This time ignore the letters `x` and `y`; but for characters `"+"` and `"-"`:

- `+` means turn the turtle left 60 degrees then move forward 20 steps
- `-` means turn the turtle right 60 degrees then move forward 20 steps

Here is some code you may find useful.

```
t = Turtle()
for c in s:
    if c == '+': t.left(60)
    elif c == '-': t.right(60)
t.forward(20)
```

Once this is working feel free to experiment. You can change the forward amount, the angles, and you can even change Rule S and Rule D.